



Contents lists available at ScienceDirect

Theoretical Computer Science

journal homepage: www.elsevier.com/locate/tcs

Priority queues with decreasing keys ☆☆☆

Gerth Stølting Brodal

Aarhus University, Department of Computer Science, Aabogade 34, 8200, Aarhus N, Denmark

ARTICLE INFO

Keywords:

Priority queue
Decreasing keys
Post-order heap
Dijkstra's algorithm

ABSTRACT

A priority queue stores a multiset of items, each item being a $\langle \text{key}, \text{value} \rangle$ pair, and supports the insertion of a new item and extraction of an item with minimum key. In applications like Dijkstra's single source shortest path algorithm and Prim-Jarník's minimum spanning tree algorithm, the key of an item can decrease over time. Usually this is handled by either using a priority queue supporting the deletion of an arbitrary item or a dedicated `DecreaseKey` operation, or by inserting the same item multiple times but with decreasing keys.

In this paper we study what happens if the keys associated with the items in a priority queue can decrease over time *without* informing the priority queue, and how such a priority queue can be used in Dijkstra's algorithm. We show that binary heaps with bottom-up insertions fail to report items with unchanged keys in correct order, while binary heaps with top-down insertions report items with unchanged keys in correct order. Furthermore, we show that skew heaps, leftist heaps, and priority queues based on linking the roots of heap-ordered trees, like pairing heaps, binomial queues and Fibonacci heaps, work correctly with decreasing keys without any modifications. Finally, we show that the post-order heap by Harvey and Zatloukal, a variant of a binary heap with amortized constant time insertions and amortized logarithmic time deletions, works correctly with decreasing keys and is a strong contender for an implicit priority queue supporting decreasing keys in practice.

1. Introduction

A priority queue is a data structure storing a multiset of items, where each item is a $\langle \text{key}, \text{value} \rangle$ pair. A basic priority queue supports the two operations `Insert` and `ExtractMin`, which insert a new item into the priority queue and extract an item with minimum key from the priority queue, respectively. A classic example of a data structure supporting these operations is the binary heap by Williams from 1964 [23]. Although many priority queues exist supporting a more comprehensive list of operations or having better asymptotic bounds, the binary heap is the standard priority queue implementation in many languages, like in Python (module `heapq`) and Java (class `java.util.PriorityQueue`). The popularity of binary heaps is due to its simplicity, it can be stored implicitly in an (extendable) array only storing the n items currently in the heap, and the number of comparisons is relatively low. Insertions require at most $\log_2 n$ comparisons, and minimum extractions at most $2 \log_2 n$ comparisons, but the number of comparisons performed are often lower in practice.

☆ This article belongs to Section A: Algorithms, automata, complexity and games, Edited by Paul Spirakis.

☆☆ Work presented at the Eleventh International Conference on Fun with Algorithms (FUN 2022) [3]. Supported by Independent Research Fund Denmark, grant 9131-00113B.

E-mail address: gerth@cs.au.dk.

URL: <https://www.cs.au.dk/~gerth/>.

<https://doi.org/10.1016/j.tcs.2024.114563>

Received 15 October 2022; Received in revised form 21 December 2023; Accepted 8 April 2024

Available online 16 April 2024

0304-3975/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Two graph algorithms fundamentally relying on efficient priority queues are Dijkstra's algorithm [7] for finding shortest paths from a source node in directed graphs with non-negative edge weights, and Prim-Jarník's algorithm [14,19] for finding the minimum spanning tree in a weighted undirected graph. Both maintain a priority queue over the nodes not included yet in the shortest path tree and minimum tree, respectively. A node in the priority queue has an associated key equal to the shortest distance to the node discovered so far and the lightest edge connecting the node to the minimum spanning tree constructed so far, respectively. For both algorithms the key of a node in the priority queue can decrease over time, which challenges the interface of the basic priority queue. One solution is to apply a more specialized priority queue, like Fibonacci heaps [11], which support a dedicated `DecreaseKey` operation – but this structure is more complicated, pointer based, and often not part of a standard library. Although theoretically worst-case superior, the overhead of supporting `DecreaseKey` only pays off when a large fraction of the edges cause a `DecreaseKey` operation to be performed. A simpler solution is to stay with a basic priority queue and just insert a node multiple times, once whenever the key decreases. This leaves outdated copies of nodes in the priority queue, but these can be skipped whenever they are extracted from the priority, since only the first extraction of a node is not outdated. Two possible implementations of this idea for Dijkstra's algorithm are shown as algorithms `Dijkstra3` and `Dijkstra4` in Fig. 1: `Dijkstra3` checks if the distance of an extracted item is outdated by comparing it to the known minimum distance, whereas `Dijkstra4` maintains a set of the visited nodes, i.e., items in the priority queue storing these nodes are outdated. In the following we do not discuss Prim-Jarník's algorithm any further.

In a typical implementation of Dijkstra's algorithm one maintains an array `dist`, where `dist[v]` is the currently shortest known distance from the source node to node v , and the items inserted into the priority queue are pairs $\langle \text{dist}[v], v \rangle$, where `dist[v]` is the key of the item. In this paper we consider adopting the idea of only storing v as an item in the priority queue *without* an explicitly associated key. The comparison between two items in the priority instead compares the current distances `dist[v]`. This will reduce the space usage for the priority queue, e.g., a binary heap only needs to store an array of node identifiers. The challenge is now that keys of inserted items can decrease over time, i.e., the ordering of the items in the priority queue changes over time and potentially invalidates the internal invariants maintained by a priority queue. In this paper we identify comparison based priority queues working correctly with such decreasing keys. In particular we show that skew heaps [21], leftist heaps [6], binomial queues [22], pairing heaps [10], Fibonacci heaps [11], and post-order heaps [13] work correctly even with decreasing keys. For binary heaps [23] we show that the standard implementation with bottom-up insertions fails to support decreasing keys, whereas binary heaps work correctly if operations are performed top-down.

1.1. Model

We define a *priority queue with decreasing keys* as follows. It stores a multiset of items, where each item is a $\langle \text{key}, \text{value} \rangle$ pair, where the key is from some totally ordered universe. Over time the key of an item can decrease an arbitrary number of times. We let the *original key* of an item refer to the key when the item was inserted, whereas the *current key* refers to the key at the current time. If the current key equals the original key we say that the item has an *unchanged key*. The priority queue is not informed when keys decrease, and whenever two items are compared by the priority queue the comparison is performed with respect to their current keys. The priority queue has no access to the original keys of the items; it can only compare two items and get the relative order of their current keys. Note that the answer to the comparison between two items can vary over time depending on how their keys decrease, and that it is possible that all items in the priority queue can have current keys strictly smaller than their original keys.

A priority queue with decreasing keys should support the following two operations:

- `Insert(x)` inserts an item $x = \langle \text{key}, \text{value} \rangle$ into the multiset of items represented by the priority queue.
- `ExtractMin()` returns an item from the priority queue with *current key* less than or equal to the *original keys* of all items in the priority queue. The returned item is removed from the multiset of items represented by the priority queue.

It follows that if `ExtractMin` returns an item with unchanged key, the item has smallest original key among all items. Furthermore, if several items in the priority queue have current key less than or equal to the smallest original key in the priority queue, then the priority queue is allowed to return any of these items, i.e., its behavior is non-deterministic. As an example, consider a priority queue with four inserted items A , B , C and D with original keys 5, 2, 6 and 4, respectively. Assume C and D have had their keys decreased to have current keys 3 and 1, respectively. The result of `ExtractMin` is now not unique. Among the four items, B has smallest original key equal to 2, i.e., any item with current key smaller than or equal to 2 is a valid result of `ExtractMin`. Then `ExtractMin` could return either B or D , since they have current keys 2 and 1, respectively. On the hand, A and C cannot be returned since they have current keys 5 and 3, respectively. In Section 2 we discuss how an implementation of Dijkstra's algorithm can benefit from the interface of a priority queue with decreasing keys.

1.2. Contributions

This paper introduces no new data structure. Only existing data structures are analyzed in the context of decreasing keys. Our contributions are:

- Section 2: We show that Dijkstra's algorithm [7] (`Dijkstra4` in Fig. 1) works correctly when using a priority queue with decreasing keys, i.e., an item in the priority queue only stores the node v instead of the pair $\langle \text{dist}[v], v \rangle$.

- Section 3: Binary heaps [23] with bottom-up insertions do not support decreasing keys, in particular we show that sorting (with interleaved key decreases) and Dijkstra’s algorithm fail on small examples.
- Section 4: Binary heaps with top-down insertions support decreasing keys, i.e., inserting a new item in a binary heap considers the ancestors of the new leaf top-down until the first ancestor is found with key greater than or equal to the new key. The central invariant used in the analysis, and also used in Sections 5 and 6, is *decreased heap order* requiring that any ancestor of a node v in a tree must store an item with current key less than or equal to the original key of v .
Without decreasing keys, bottom-up and top-down insertions cause the nodes of the resulting heaps to store identical keys, but for random insertions the number of comparisons increases from average $O(1)$ [18] to $\Theta(\log n)$. In Section 7 we present an experimental comparison of the two variants of a binary heap, and in particular the overhead introduced by performing insertions top-down.
- Section 5: Skew heaps [21], leftist heaps [6], pairing heaps [10], binomial queues [22], and Fibonacci heaps [11] work correctly with decreasing keys.
- Section 6: The post-order heap by Harvey and Zatloukal [13] supports decreasing keys. The post-order heap is a simple implicit heap based on binary heaps supporting insertions in amortized constant time and extractions in amortized logarithmic time (like, e.g., binomial queues). In Section 7 our experimental evaluation shows that the post-order heap is a strong contender for an efficient implicit priority queue supporting decreasing keys.
- Section 7: We supplement our theoretical results with an experimental evaluation of priority queues supporting decreasing keys and compare the number of key comparisons performed to sort and to run Dijkstra’s algorithm on cliques.

1.3. Related work

The literature on priority queues is comprehensive, see, e.g., the survey by Brodal [2]. Fibonacci heaps [11] support `DecreaseKey` in amortized constant time and their discovery initiated the study of data structures supporting efficient `DecreaseKey` operations. Subsequently, e.g., relaxed heaps [8] were introduced, which support `DecreaseKey` in worst-case constant time. In this paper we focus on simpler data structures, not supporting dedicated `DecreaseKey` operations. Many priority queues can be extended to support an arbitrary `Remove` operation, by having a separate index keeping track of where each item is stored in the data structure. This introduces a space overhead for the index and a time overhead to keep the index updated, e.g., swapping two items in a binary heap requires two entries in the index to be updated. If only `Insert` and `ExtractMin` need to be supported, many simple constructions exist—a few commonly used are mentioned and evaluated in this paper. A special interesting class of priority queues are those that can be stored in a single array containing the items, known as *implicit* priority queues. The classic example is the binary heap [23], but other examples are, e.g., the implicit binomial trees by Carlsson, Munro and Poblette [4], and the post-order heap by Harvey and Zatloukal [13] that is our focus in Section 6. Many priority queues maintain a forest of trees of sizes corresponding to digits in the binary or skew binary representation of the total number of items stored. Elmasry, Jensen and Katajainen [9] give an overview of this relationship for various constructions.

1.4. Background

The motivation for studying priority queues with decreasing keys arose from experience with undergraduate students having problems translating Dijkstra’s shortest path algorithm into correct Java programs based on the description in the standard text book by Cormen et al. [5, Section 22.3]. Students are challenged by the fact that the priority queue implementation supported by the Java standard library¹ does not support `DecreaseKey`, the `Remove` operation requires linear time, and the ordering of values is provided using a comparator (or the natural ordering of the values). A priority queue with decreasing keys allows Dijkstra’s algorithm to store node identifiers only in the priority queue and using a comparator to compare two nodes by comparing their currently best known distances—the Java solution many students implement, but fails since their priority queue does not support decreasing keys.

2. Dijkstra’s algorithm with decreasing keys

Assume we are given a directed graph $G = (V, E)$ with non-negative edge weights δ and a source node $s \in V$, and we want to compute the shortest distance from s to all nodes in the graph. This problem can be solved using Dijkstra’s algorithm [7]. The basic idea of Dijkstra’s algorithm is to visit nodes in increasing distance from s . For each node v not visited yet its currently known distance $\text{dist}[v]$ is stored in an array dist , i.e., the distance to v along paths only containing v and already visited nodes. The next node to visit is a node u not visited so far and with smallest $\text{dist}[u]$ value. When visiting a node u we *relax* along its outgoing edges (u, v) by performing the update $\text{dist}[v] := \min(\text{dist}[v], \text{dist}[u] + \delta(u, v))$. To obtain an efficient solution, the set of nodes not visited yet are stored in a priority queue, with $\text{dist}[v]$ as the key of v . Fibonacci heaps [11] provide a dedicated `DecreaseKey` operation to update (decrease) the key of a node whenever the known distance to a node decreases. Using a Fibonacci heap Dijkstra’s algorithm can be implemented as shown in `Dijkstra1` in Fig. 1 obtaining running time $O(|E| + |V| \log |V|)$. If no `DecreaseKey` is available, but an arbitrary item can be removed by a `Remove` operation, we can simulate `DecreaseKey` by first removing the node using `Remove` and then reinserting the node with its smaller distance as key using `Insert` as shown in `Dijkstra2` in Fig. 1. If `Remove`

¹ <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/PriorityQueue.html>.

```

proc Dijkstra1( $V, E, \delta, s$ )
   $dist[v] = +\infty$  for all  $v \in V \setminus \{s\}$ 
   $dist[s] = 0$ 
  Insert( $Q, \langle dist[s], s \rangle$ )
  while  $Q \neq \emptyset$  do
     $\langle d, u \rangle = \text{ExtractMin}(Q)$ 
    for  $(u, v) \in E \cap (\{u\} \times V)$  do
      if  $dist[u] + \delta(u, v) < dist[v]$  then
         $dist[v] = dist[u] + \delta(u, v)$ 
        if  $v \in Q$  then
          DecreaseKey( $Q, v, dist[v]$ )
        else
          Insert( $Q, \langle v, dist[v] \rangle$ )
  return  $dist$ 

proc Dijkstra2( $V, E, \delta, s$ )
   $dist[v] = +\infty$  for all  $v \in V \setminus \{s\}$ 
   $dist[s] = 0$ 
  Insert( $Q, \langle dist[s], s \rangle$ )
  while  $Q \neq \emptyset$  do
     $\langle d, u \rangle = \text{ExtractMin}(Q)$ 
    for  $(u, v) \in E \cap (\{u\} \times V)$  do
      if  $dist[u] + \delta(u, v) < dist[v]$  then
         $dist[v] = dist[u] + \delta(u, v)$ 
        if  $v \in Q$  then
          Remove( $Q, v$ )
          Insert( $Q, \langle dist[v], v \rangle$ )
  return  $dist$ 

proc Dijkstra3( $V, E, \delta, s$ )
   $dist[v] = +\infty$  for all  $v \in V \setminus \{s\}$ 
   $dist[s] = 0$ 
  Insert( $Q, \langle dist[s], s \rangle$ )
  while  $Q \neq \emptyset$  do
     $\langle d, u \rangle = \text{ExtractMin}(Q)$ 
    if  $d = dist[u]$  then
      for  $(u, v) \in E \cap (\{u\} \times V)$  do
        if  $dist[u] + \delta(u, v) < dist[v]$  then
           $dist[v] = dist[u] + \delta(u, v)$ 
          Insert( $Q, \langle dist[v], v \rangle$ )
  return  $dist$ 

proc Dijkstra4( $V, E, \delta, s$ )
   $dist[v] = +\infty$  for all  $v \in V \setminus \{s\}$ 
   $dist[s] = 0$ 
   $visited = \emptyset$ 
  Insert( $Q, \langle dist[s], s \rangle$ )
  while  $Q \neq \emptyset$  do
     $\langle d, u \rangle = \text{ExtractMin}(Q)$ 
    if  $u \notin visited$  then
       $visited = visited \cup \{u\}$ 
      for  $(u, v) \in E \cap (\{u\} \times V)$  do
        if  $dist[u] + \delta(u, v) < dist[v]$  then
           $dist[v] = dist[u] + \delta(u, v)$ 
          Insert( $Q, \langle dist[v], v \rangle$ )
  return  $dist$ 

```

Fig. 1. Four variations of Dijkstra's algorithm for the single source shortest path problem on a digraph with nodes V , edges E , edge weights δ , and source node s . The main result of this paper is that Dijkstra₄ still works correctly if we adopt a priority with decreasing keys.

takes logarithmic time, the resulting running time is $O(|E| \log |V|)$. For sparse graphs, i.e., $|E| = O(|V|)$, the two running times are asymptotically identical. A priority queue supporting efficient **DecreaseKey** and/or **Remove** operations must be able to locate an item in the priority queue, and for implicit heap structures this is typically achieved by maintaining a second array that maps a node identifier to a location in the heap.

Here we consider a simpler implementation using a binary heap only supporting **Insert** and **ExtractMin**, but also achieving running time $O(|E| \log |V|)$. Whenever a shorter distance is found to a node v , we insert the item $\langle dist[v], v \rangle$ into the heap, i.e., the same node v can be inserted multiple times, but with decreasing keys. All instances of v in the heap, except for the one with key equal to the current $dist[v]$, are outdated and should be ignored/skipped when extracted from the heap. We can identify nodes to be skipped by either comparing the extracted distance with the currently best known distance or by keeping a set of all visited nodes, e.g., a bit-vector $visited$. Algorithms Dijkstra₃ and Dijkstra₄ in Fig. 1 contain the pseudo code for these implementations of Dijkstra's algorithm. Note that Dijkstra₃ and Dijkstra₄ perform exactly the same computations, the only difference is the approach used to identify outdated items extracted from the priority queue. We will not argue further about the correctness of these variations of Dijkstra's algorithm (leaving outdated items in the priority queue is also a common approach to external memory algorithms for the single source shortest path problem, see, e.g., [15, Section 4.2]).²

Crucial to the above implementations of Dijkstra's algorithm is that each item we insert into the heap is a pair $\langle dist[v], v \rangle$, where the key $dist[v]$ is fixed when inserted. These keys require space in the heap, that could be tempting to save. The (potentially dangerous) idea is now: *Skip storing the keys explicitly in the items and instead use the current value $dist[v]$ as the current key for all items storing v in the heap.*

In the following we argue that Dijkstra₄ still works correctly if we adopt a priority with decreasing keys, e.g., those in Sections 4–6. The only changes to Dijkstra₄ is that **Insert** should only take the node to insert (without the distance), and **ExtractMin** does not return the key/distance d (that anyway was not used by Dijkstra₄ after the item was extracted), and whenever the priority queue compares the keys of two nodes u and v it compares the currently known distances $dist[u]$ and $dist[v]$.

The invariant maintained by the algorithm is that only nodes v with $dist[v] < +\infty$ are stored in the priority queue, and for all nodes with $dist[v] < +\infty$ and $v \notin visited$, the priority queue contains an item containing v with unchanged key equal to the current $dist[v]$. This is true since we insert an item containing v with original key $dist[v]$ whenever $dist[v]$ has decreased.

² In the worst-case the priority queue stores $O(|E|)$ items, but this can be reduced to $O(|V|)$ items by rebuilding the heap whenever it contains $> (1 + \epsilon)|V|$ items, for some constant $\epsilon > 0$, where all outdated items are removed. The time for rebuilding the heap can be charged to the removed items, i.e., the asymptotic running time remains unchanged.

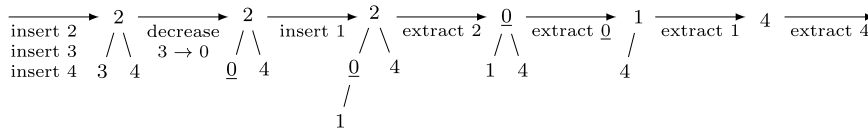


Fig. 2. Binary heap failing to sort $\langle 2, 3, 4, 1 \rangle$ if 3 is decreased to 0 (underlined) before inserting 1.

Whenever an item with a node v is extracted from the priority queue, we have three cases: i) $v \in \text{visited}$, ii) $v \notin \text{visited}$ and the current key of the item equals the original key, and iii) $v \notin \text{visited}$ and current key of the item is less than the original key. In case i) we extract a node that has already been visited, and therefore should be skipped. In case ii) we extract an item with current key equal to its original key. Since the priority queue guarantees that the current key of the extracted item is less than or equal to all original keys stored in the priority queue, the item has minimum original key among all items in the priority queue. From the invariant it follows that v has smallest $\text{dist}[v]$ value among all nodes not visited yet—as required by Dijkstra’s algorithm. Finally, in case iii) an item is extracted containing a node v not visited yet and with current key less than its original key. By the priority queue specification, its current key, i.e., $\text{dist}[v]$, is less than all original keys in the priority queue, in particular those stored with unchanged keys. Since $v \notin \text{visited}$ the invariant implies there must exist another item in the priority queue storing v with original and current key equal to $\text{dist}[v]$ —i.e., v is a node not visited yet with minimum dist value as required by Dijkstra’s algorithm. It follows that a priority queue supporting decreasing keys extracts unvisited nodes in increasing order of distance as required by Dijkstra’s algorithm.

3. Binary heaps and binary search trees fail with decreasing keys

In this section we show how binary heaps with bottom-up insertions [23] fail to support decreasing keys on two simple examples: Sorting and Dijkstra’s single source shortest path algorithm. We also show that binary search trees fail to support decreasing keys.

We first briefly recall the structure of a binary heap. A binary heap stores n items in an array $H[1..n]$, that can be viewed as a binary tree where node i has children $2i$ and $2i + 1$.³ The items are stored such that *heap order* is satisfied, i.e., the key of item $H[i]$ is greater than or equal to the key of its parent $H[\lfloor i/2 \rfloor]$. A bottom-up insertion places the new item as the last item in H and repeatedly swaps it with its parent (sift-up) as long as its key is less than the parent’s *current* key. A minimum extraction returns the item at the root $H[1]$, and moves the last item x from $H[n]$ to $H[1]$, and sifts-down x by repeatedly swapping x with the item with smallest key among its children until no child stores an item with key less than x .

3.1. Sorting

Consider sorting items by inserting them into a priority queue and then extracting them in increasing key order. If an item gets its key decreased during the sequence of operations, a priority queue with decreasing keys guarantees that the items with unchanged keys are still reported in increasing key order. A binary heap with bottom-up insertions fails to do so when inserting four items with keys 2, 3, 4 and 1, and where key 3 is decreased to 0 before inserting 1, as illustrated in Fig. 2. Recall that the heap is not informed when keys decrease, causing the current keys to violate heap order. Instead of reporting the items with unchanged keys in order $\langle 1, 2, 4 \rangle$ they are reported in order $\langle 2, 1, 4 \rangle$. Note that when inserting 1 as the last leaf, it is compared to its parent with current key 0 (but original key 3), where the sift-up terminates, and incorrectly leaves 1 in the subtree of 2, causing 2 to be the first item extracted by `ExtractMin`.

3.2. Dijkstra’s algorithm

Fig. 3 shows a graph with 5 nodes and 7 edges, where Dijkstra’s algorithm (`Dijkstra4`) fails when using a binary heap with bottom-up insertions and using $\text{dist}[v]$ as the current key for node v . Whenever a smaller distance to a node is discovered, the node is inserted with the new distance as its original key. The previously inserted copies of the node (if any) get their current keys decreased to the new smaller distance, like b where b_2 is the copy of b in the heap with original key 5 and current key 2. Note that when inserting b_2 , b_2 blocks b from sifting up to the root, causing the algorithm to incorrectly visit node c before node b . The result is that the distance to node d is incorrectly computed to be 5 instead of 4.

It should be noted that if we skipped the test $u \notin \text{visited}$, the algorithm would compute the correct distances by revisiting nodes whenever a shorter distance to a node has been discovered. This is against the principle idea of Dijkstra’s algorithm to only visit nodes once in increasing order of distance from the source, so that each edge is considered at most once. See the discussion in Section 7.3 on Fig. 9 for further details.

3.3. Binary search trees

Binary search trees can also be used as a priority queue (both balanced search trees, like AVL-trees [1] and red-black trees [12], and unbalanced binary search trees). Recall that a binary search tree stores the items left-to-right in sorted key order, i.e., the item

³ In the paper we assume arrays start at index 1. In our implementation we adapt to Python lists, which start at index 0.

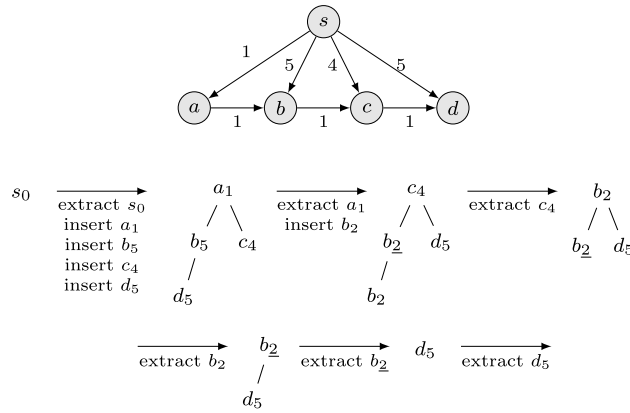


Fig. 3. Execution of Dijkstra's single source shortest path algorithm (Dijkstra_4), using dist as keys and a binary heap with bottom-up insertions, incorrectly computing the distance to d as 5. Top is input graph and below the content of the binary heap. Subscripts are keys, and underlined keys are decreased keys.

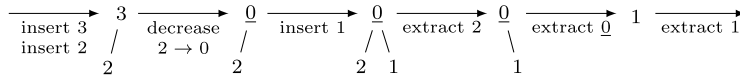


Fig. 4. Binary search trees fail to support decreasing keys, when 3 is decreased to 0 (underlined) before inserting 1.

with minimum key is stored at the leftmost node on the left spline in the tree. Insertions are performed top-down by comparing with the root, and recursively inserting in left subtree if the key at the root is larger than the new key. Otherwise, the new item is recursively inserted in the right subtree. The insertion terminates at an empty leaf, where the new item is inserted.

Fig. 4 shows how a binary search fails to support decreasing keys: We insert keys 3, 2, 1, but before inserting 1 the key 3 is decreased to 0, i.e., 1 will be inserted as the rightmost node. When performing the first ExtractMin operation, the leftmost key is 2, that will be returned as being the smallest key by a binary search tree (a binary search tree never needs to compare keys when extracting the item with minimum key). This is incorrect, since 1 is a smaller original key in the tree. Both 0 and 1 would have been correct, since they are both less than or equal to 2, that is the smallest original key.

4. Binary heaps with top-down insertions

In a binary heap with top-down insertions $\text{Insert}(x)$ creates a new empty leaf at position n , and items on the path from the root to the new leaf are compared with x top-down until the first node u is found with current key greater than or equal to the new key. The items on the path from u to the new leaf are sifted one level down and item x is inserted in node u . The ancestor at depth $d = 0, \dots, \lfloor \log_2 n \rfloor$ is node $\lfloor n/2^{\lfloor \log_2 n \rfloor - d} \rfloor$. If keys are distinct and do not decrease, then top-down insertions and bottom-up insertions yield identical structures.

In the following we let key_{org} denote an original key and key_{cur} a current key. For a tree structure, where each node stores an item, we say that the tree satisfies the *decreased heap order* if and only if $u.\text{key}_{\text{cur}} \leq v.\text{key}_{\text{org}}$ for all ancestors u of a node v . Note that if decreased heap order is satisfied, then the item at the root of a tree satisfies the conditions to be returned by ExtractMin , and decreased heap order remains satisfied when current keys decrease.

During $\text{Insert}(x)$ a node v can only get one new ancestor, namely x . This happens when x is compared with an ancestor u of v with the result $x.\text{key}_{\text{cur}} \leq u.\text{key}_{\text{cur}}$. Since before the insertion decreased heap order ensures $u.\text{key}_{\text{cur}} \leq v.\text{key}_{\text{org}}$, we have $x.\text{key}_{\text{cur}} \leq v.\text{key}_{\text{org}}$ and the tree satisfies decreased heap order after the insertion.

An ExtractMin operation returns the root of the tree. By the decreased heap order the item returned has current key less than or equal to all original keys in the tree. Before returning the answer, the last item x is moved to the root and sifted down, where x is swapped with the item at the child with smallest current key until the items of both children have current key $\geq x.\text{key}_{\text{cur}}$. Whenever two siblings v and w are compared, say $v.\text{key}_{\text{cur}} \leq w.\text{key}_{\text{cur}}$, we have two cases. If x becomes the parent of v and w , since $x.\text{key}_{\text{cur}} \leq v.\text{key}_{\text{cur}} \leq w.\text{key}_{\text{cur}}$, then $x.\text{key}_{\text{cur}}$ is less than or equal to the original keys in all nodes below x , since either $v.\text{key}_{\text{cur}}$ or $w.\text{key}_{\text{cur}}$ was so before the operation. Otherwise, v becomes the parent of w , and x and all nodes in w 's subtree get v as a new ancestor. Since $v.\text{key}_{\text{cur}} \leq w.\text{key}_{\text{cur}}$, then $v.\text{key}_{\text{cur}}$ is less than or equal to all original keys in w 's subtree, and $v.\text{key}_{\text{cur}} \leq x.\text{key}_{\text{cur}} \leq w.\text{key}_{\text{org}}$. It follows that after ExtractMin the tree still satisfies decreased heap order.

5. Existing heaps supporting decreasing keys

In the following we argue that the internal workings of several existing priority queues ensure decreased heap order to be maintained in the presence of decreasing keys.

```

proc merge( $u = \langle u_0, u_1, \dots \rangle$ ,  $v = \langle v_0, v_1, \dots \rangle$ )
  if  $u = \emptyset$  then
    return  $v$ 
  if  $v = \emptyset$  then
    return  $u$ 
  if  $u_0.key_{cur} \leq v_0.key_{cur}$  then
    return  $\langle u_0 \rangle + \text{merge}(\langle u_1, \dots \rangle, v)$ 
  else
    return  $\langle v_0 \rangle + \text{merge}(u, \langle v_1, \dots \rangle)$ 

```

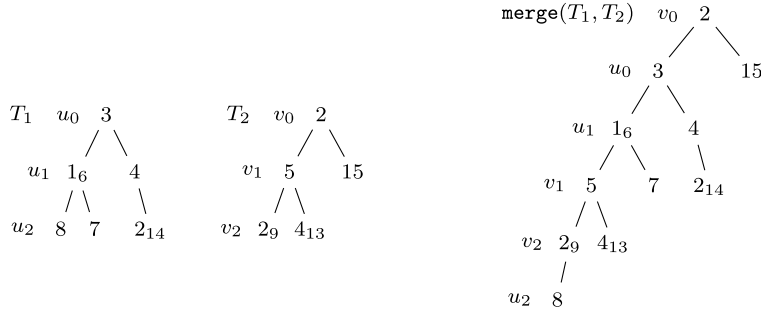


Fig. 5. Top-down merging two paths u and v . In the example values are current keys and subscripts original keys (subscripts are omitted if current and original keys are equal).

5.1. Skew heaps and leftist heaps

Skew heaps [21] and leftist heaps [6] support **Insert** and **ExtractMin** on a heap storing n items in time $O(\log n)$, where the time for skew heaps is amortized and for leftist heaps worst-case. Both data structures represent a priority queue by a (decreased) heap ordered binary tree, and support **ExtractMin** by removing the root and returning its item. All other structural changes consist of merging top-down two root-to-leaf paths in two (decreased) heap ordered binary trees, and potentially swapping the left and right subtrees at nodes of the resulting tree. Swapping the left and right subtrees of a node does not change ancestor relationships, i.e., does not affect decreased heap order. To argue that the two data structures maintain decreased heap order, we only need to argue that merging two paths ensures that the resulting tree satisfies decreased heap order.

Normally the keys along the two paths would appear in increasing key order, but this is not necessarily the case when keys can decrease (in fact the current keys can appear in any order). Assume the nodes along the two root-to-leaf paths to be merged are $\langle u_0, u_1, u_2, \dots \rangle$ and $\langle v_0, v_1, v_2, \dots \rangle$, and the merging is performed top-down recursively as in Fig. 5. If u_i ends up before v_j , i.e., u_i is a new ancestor of v_j , then there exists $j' \leq j$ where u_i and $v_{j'}$ have been compared with the outcome $u_i.key_{cur} \leq v_{j'}.key_{cur}$. Since by assumption $v_{j'}.key_{cur} \leq v_j.key_{org}$, it follows that u_i satisfies decreased heap order with v_j and all its descendants. It follows that the resulting tree after merging two root-to-leaf paths in two decreased heap ordered trees also satisfies decreased heap order.

5.2. Pairing heaps, binomial queues, and Fibonacci heaps

Many priority queues represent a priority queue by one or more heap ordered trees of arbitrary degree. Pairing heaps [10], binomial queues [22], and Fibonacci heaps [11] are examples of such priority queues supporting **Insert** in amortized time $O(1)$ and **ExtractMin** in amortized time $O(\log n)$. Here we prove that these data structures maintain decreased heap order when used with decreasing keys. Pairing heaps only represent a priority queue by a single tree, whereas binomial queues and Fibonacci heaps maintain a forest, and the item removed by **ExtractMin** is the root with minimum current key, i.e., the returned item has current key less than or equal to all original keys in all trees. Nodes only get new ancestors when two roots are *linked*, that makes the root v with greatest current key a child of the root u with smallest current key. Since $u.key_{cur} \leq v.key_{cur}$ and any node w in the tree rooted at v has $v.key_{cur} \leq w.key_{org}$, it follows that $u.key_{cur} \leq w.key_{org}$, i.e., the linked tree satisfies decreased heap order. It follows that the resulting heaps satisfy decreased heap order.

For Fibonacci heaps, the operations **DecreaseKey** replaces the key of an item by a smaller key. In our context this corresponds to lowering the original key. In the context of decreasing keys, it is important that **DecreaseKey** is implemented to always cut the edge from the node to its parent (without comparing with the current key of the parent, since that could have been decreased arbitrarily), and to add the node as a new root to the forest.

6. Post-order heap

In practice, binary heaps with bottom-up insertions often benefit from the fact that insertions do not sift-up items far in the tree. With top-down insertions this property cannot be exploited. In this section we consider the post-order heap by Harvey and

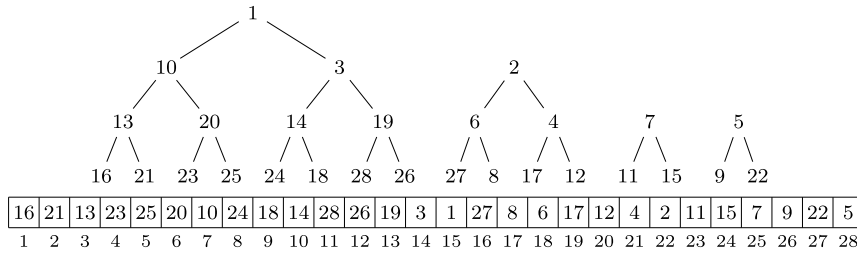


Fig. 6. Top: A post-order heap storing 28 items in four trees of size 15, 7, 3 and 3. Bottom: The implicit post-order layout in a single array.

```

proc Insert( $x$ )
  push( $H, x$ )
  if  $|S| \geq 2$  and  $S[|S|] = S[|S| - 1]$  then
     $size = \text{pop}(S) + \text{pop}(S) + 1$ 
    push( $S, size$ )
    Heapify( $H, size$ )
  else
    push( $S, 1$ )

proc Heapify( $i, size$ )
  if  $size > 1$  then
     $size = \lfloor size/2 \rfloor$ 
     $right = i - 1$ 
     $left = right - size$ 
     $smallest = H[left] < H[right] ? left : right$ 
    if  $H[smallest] < H[i]$  then
      swap  $H[i]$  and  $H[smallest]$ 
      Heapify( $smallest, size$ )

proc ExtractMin()
   $min = +\infty$ 
   $i = |H|$ 
  for  $j = 1$  to  $|S|$  do
     $size = S[|S| - j + 1]$ 
    if  $H[i] < min$  then
       $min = H[i]$ 
       $i_{min} = i$ 
       $size_{min} = size$ 
     $i = i - size$ 
   $size = \lfloor \text{pop}(S)/2 \rfloor$ 
  if  $size > 0$  then
    push( $S, size$ )
    push( $S, size$ )
   $x = \text{pop}(H)$ 
  if  $i_{min} < |H|$  then
     $H[i_{min}] = x$ 
    Heapify( $i_{min}, size_{min}$ )
  return  $min$ 

```

Fig. 7. Post-order heap operations, where H stores the items and S is a list of tree sizes.

Zatloukal [13]. In our experiments (see Section 7) it appears to be a strong contender for an efficient implicit priority queue supporting decreasing keys, which is why we consider it in more detail in this section. Harvey and Zatloukal [13] did an experimental comparison of C# implementations of post-order heaps and binary-heaps with bottom-up insertions and found that post-order heaps had faster insertions but slower deletions.

A post-order heap consists of a forest of complete heap ordered binary trees, where all trees have distinct size, except for possibly the two trees of smallest size. Since a complete binary tree has size $2^i - 1$, for some i , the number of trees of each size corresponds to the digits in the skew binary number representation of n . Myers [17] proved that the set of tree sizes is unique for a given n . The trees are laid out consecutively in a single array H in decreasing size order, and each tree in post-order. For a subtree of size s with root $H[i]$, the subtree is stored in $H[i - s + 1, i]$, the subtrees at the children have size $\lfloor s/2 \rfloor$, the right child is $H[i - 1]$, and the left child is $H[i - 1 - \lfloor s/2 \rfloor]$. See Fig. 6.

Insert(x) inserts the item x as the last item of H . If the last two trees had different size, x becomes a tree of size one. Otherwise, the last two trees of size s together with x are combined to a new tree of size $2s + 1$ with x as the root, and we apply the sift-down operation Heapify($|H|, 2s + 1$) (where the first argument is the node position, and the second argument is the size of the subtree). Except for node indexing, Heapify is implemented as for binary heaps. To support decreasing keys, it is crucial that Heapify is performed top-down. ExtractMin() identifies the root min with minimum (current) key to return, and removes the root x from the rightmost tree (causing the two subtrees to become new trees). If $x \neq min$, x replaces the root min , and x is sifted down using Heapify. Pseudo-code for the operations is given in Fig. 7 (in [13] it is discussed how the list of tree sizes S of length $O(\log n)$ can be stored using only $O(\log n)$ bits). Since a post-order heap structurally is just a collection of binary heaps updated only using top-down Heapify, the discussion from Section 4 carries over to prove that post-order heaps support decreasing keys.

If we consider the worst-case number of comparisons for binary heaps with top-down insertions, then Insert uses at most $\lfloor \log_2 n \rfloor$ comparisons whereas ExtractMin at most $2\lfloor \log_2 n \rfloor$ comparisons. Harvey and Zatloukal [13] proved that post-order heaps support Insert in amortized time $O(1)$ and ExtractMin in amortized time $O(\log n)$. The worst-case number of comparisons for operations on post-order heaps is not competitive with binary heaps, since in the worst-case Insert performs Heapify on a tree containing all items, i.e., requiring at most $2\lfloor \log_2 n \rfloor$ comparisons, and ExtractMin first must find the root with minimum value, and then perform Heapify on this tree, requiring at most $3\lfloor \log_2 n \rfloor$ comparisons.

It appears that adding support for decreasing keys to an implicit priority queue with the ExtractMin/Insert interface comes with a significant overhead. That the overhead in a sequence of ExtractMin and Insert operations can be smaller, follows from the example of sorting by performing n Insert operations followed by n ExtractMin operations (without using the support for decreasing keys). The above worst-case bounds for the operations implies an upper bound on sorting using a post-order heap of

$5n \lfloor \log_2 n \rfloor$ comparisons, whereas using a binary heap requires at most $3n \lfloor \log_2 n \rfloor$ comparisons. But for sorting we can derive a better bound for post-order heaps. During the n insertions at most $n/2^h$ roots are created at height h , each requiring $2h$ comparisons for a sift-down, causing a total of at most $\sum_{h=0}^{\infty} 2h \cdot n/2^h \leq 4n$ comparisons for insertions. Furthermore, during the sequence of minimum extractions the average number of trees is $\frac{1}{2} \log_2 n + O(1)$, causing the total number of comparisons for finding the minimum roots to be at most $\frac{1}{2} n \log_2 n + O(n)$. Together with the upper bound of $2 \lfloor \log_2 n \rfloor$ comparisons for each `Heapify` caused by an `ExtractMin` gives a total bound of $2.5n \log_2 n + O(n)$ comparisons for sorting using a post-order heap. An experimental comparison of post-order heaps with binary heaps with top-down insertions is done in Section 7 (Fig. 8).

7. Experimental evaluation

The previous sections state that many priority queue data structures work in the setting with decreasing keys. Any implementation of these data structures will also work if explicit keys can be removed and handled by implicit decreasing keys, e.g., as in Dijkstra's algorithm implemented by a comparator accessing the array `dist`. The worst-case analysis of these data structures carries over to the setting with decreasing keys and the worst-case running time analysis of Dijkstra's algorithm remains unchanged, though in practice the picture could be different. In particular items to be skipped (i.e., with decreased keys) in Dijkstra's algorithm can be extracted earlier when allowing decreasing keys.

We implemented in Python 3.12 various priority queues and Dijkstra's algorithm to have code as close as possible to pseudo code, and to focus on measuring counts that were hardware, language, and compiler independent.⁴ A priority queue was implemented as a class with `extract_min` and `insert` methods to update the priority queue, and a method `empty` to test for emptiness. Items are compared using the `<` operator, i.e., using the `__lt__` method of the items. Finally, each priority queue has a method `validate` to check the structural integrity of its current content, e.g., a recursive traversal checking heap order, number of children, and balance conditions. The experimental evaluation was done on a HP EliteBook 840 G8 laptop (Intel i7-1165G7 CPU, 16 GB RAM) running Python 3.12.0 under Windows 11.

The following priority queues were implemented: Skew heaps [21], leftist heaps [6], binomial queues [22], pairing heaps [10], post-order heaps [13], and binary heaps [23] with bottom-up and top-down insertions. Finally, we made a wrapper class around Python's builtin module `heapq` that is a C implementation⁵ of binary heaps with bottom-up insertions. We did not implement Fibonacci heaps [11], since we do not consider dedicated `DecreaseKey` operations, and without `DecreaseKey` Fibonacci heaps are identical to binomial queues.

We considered four versions of binary heaps, where the first two do not support decreasing keys: `BinaryHeap` is a standard binary heap with bottom-up insertions and top-down heapify to sift-down the new root value during minimum extractions. `BinaryHeapHeapifyBottomup` improves typical performance by letting heapify first recursively pull up the child with smallest (current) key until an empty leaf is created, where the item from the last leaf is inserted and sifted up (our experiments confirm that module `heapq` implements this idea). The last two variants support decreasing keys. `BinaryHeapTopdown` supports insertions by performing comparisons top-down along the root-to-new-leaf path, until the first node is reached with greater or equal (current) key. The new item is inserted in this node, and all remaining nodes on the path to the last leaf are sifted one level down. `BinaryHeapTopdownHeapify` has a slightly naïver insertion implementation, where all items on the root-to-new-leaf path are sifted one level down, and the new item is inserted at the root and sifted down by `heapify`.

Experiments were parameterized by the priority queue class to be tested, to ensure identical testing overhead for the different classes. In our experiments we have measured the number of comparisons performed, various counts and running time. All priority queues were tested with exactly the same set of inputs. Data structures not supporting decreasing keys (`Heapq`, `BinaryHeap` and `BinaryHeapHeapifyBottomup`) are shown with dashed curves. In all plots `Heapq` and `BinaryHeapHeapifyBottomup` have identical curves, except for the time for sorting in Fig. 8(e).

7.1. Correctness of implementation

To have some evidence for the correctness of our implementations, we performed two simple sorting tests: The first checks if `Insert` and `ExtractMin` work correctly if no keys decrease, and the second checks if decreasing keys are supported. The second stress test was in fact used to identify which priority queues supported decreasing keys, before knowing if they did so, directing the search for the formal arguments presented in Sections 3–6.

7.1.1. Sorting

Each priority queue implementation was used to sort various input sequences of n numbers, $1 \leq n \leq 1000$, with input being the increasing sequence $1, \dots, n$, the decreasing sequence $n, \dots, 1$, a uniform random permutation of $1, \dots, n$, and n uniformly selected random integers from $1, \dots, n$ (with possible repetitions). Only the random integer inputs can contain duplicates, where the expected number of distinct integers among n integers is $n(1 - (1 - 1/n)^n) \approx n(1 - 1/e) \approx 0.632n$. Each input was first inserted using n calls to `insert` followed by calls to `extract_min` until `empty` returned true. The output was checked against Python's builtin function `sorted`. After each update the priority queue's `validate` method was called to check for internal integrity.

⁴ Python source code used for experiments and data visualized in figures is available at <https://www.cs.au.dk/~gerth/papers/fun22code.zip>.

⁵ https://github.com/python/cpython/blob/master/Modules/_heapqmodule.c.

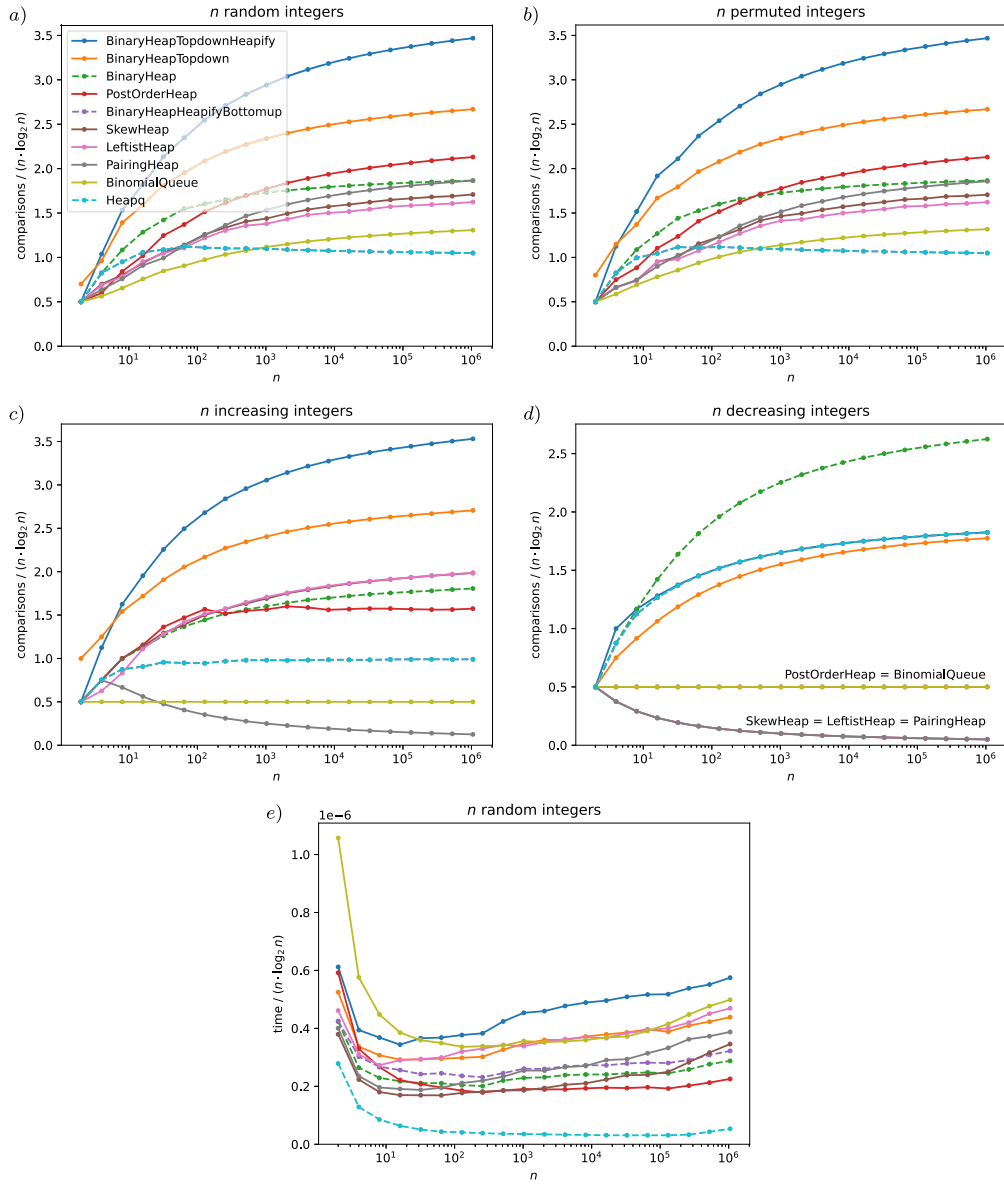


Fig. 8. Sorting n integers using various priority queues.

7.1.2. Decrease key support

Each priority queue implementation was tested for the support of decreasing keys by performing the following experiment 1000 times with $n = 100$: Insert a random permutation of $1, \dots, n$ using n calls to `Insert`, followed by n calls to `ExtractMin`. After each operation, with probability $1/2$ a random inserted item has its key decreased to zero. At the end it is checked if all items with unchanged key were reported in sorted order. As expected by the theory, all priority queues not supporting decreasing keys failed this stress test, whereas the others succeeded on all inputs

7.2. Sorting performance

We evaluated the performance of the different priority queues by using them to sort n integers for different n (powers of two, $n \leq 2^{20}$) and different input distributions: increasing sequences, decreasing sequences, uniformly permuted sequences, uniformly selected random integers from $1, \dots, n$. Each input was run at least 3 times and until at least 0.2 seconds were passed. For random inputs 10 different inputs were generated and the average computed. The measured average number of comparisons for each input size and type is shown in Fig. 8(a, b, c, d). In the plots we on the y-axis have number of comparisons performed divided by $n \log_2 n$, i.e., the theoretical asymptotic worst-case bound of sorting.

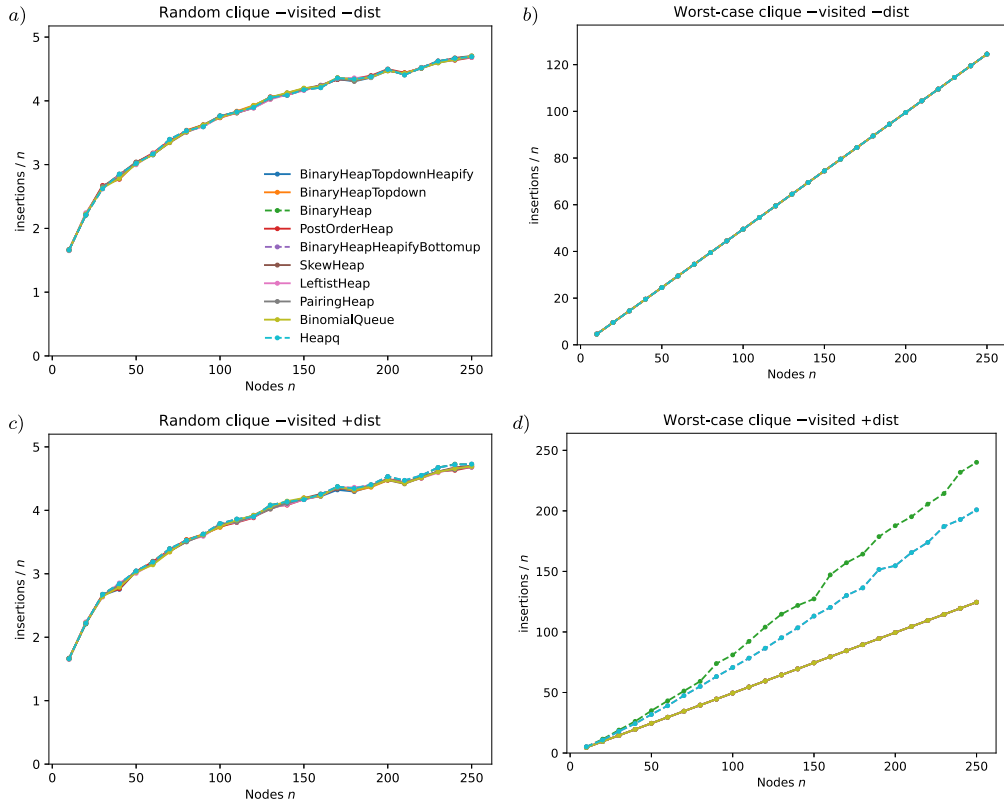


Fig. 9. Dijkstra, number of insertions into the priority queue when not using a visited bit-vector. (c, d) Using *dist* as key with a heap not supporting decreasing keys (dash curves) can cause multiple visits to the same node to discover shorter distances.

That Python’s builtin `Heapq` module is equivalent to the Python implementation `BinaryHeapHeapifyBottomup` follows from the plots, where the two priority queues achieve coinciding number of comparisons (it was checked that the number of comparisons performed were identical). Among the implicit constructions based on binary heaps we have a clear ordering (except for decreasing sequences) where `BinaryHeapHeapifyBottomup` (and `Heapq`) performs the fewest comparisons, followed by `BinaryHeap`, `BinaryHeapTopdown` and `BinaryHeapTopdownHeapify`, where only the last two support decreasing keys. In all cases the implicit `PostOrderHeap` achieves a better performance than the other implicit binary heaps supporting decreasing keys. In particular we see that `PostOrderHeap` performs about $\frac{1}{2}n \log_2 n$ fewer comparisons as `BinaryHeapTopdown` for random input, as expected by the discussion in Section 6. The only priority queues supporting decreasing keys that achieve significant better bounds on the number of comparisons are all pointer based (`SkewHeap`, `LeftistHeap`, `PairingHeap` and `BinomialQueue`). This leaves the post-order heap as a strong contender for an implicit priority queue supporting decreasing keys—at least with respect to comparisons.

With respect to running times the builtin `Heapq` consistently achieved the best time-wise performance, which is not surprising since it is implemented in C, whereas the other implementations are clearly penalized by the overhead of Python being interpreted. Running times for random input is shown in Fig. 8(e). The increased running time for less than ten elements for all priority queues is likely due to the overhead of the timing framework. Interestingly, for large random inputs the running time of post-order heaps is only outmatched by the builtin `heapq`, although the overhead of using Python makes the results less conclusive. Since the motivation for studying priority queues with decreasing keys is to achieve space efficiency by avoiding storing keys with the items, the implicit post-order heap appears to be a good choice of data structure in this context.

7.3. Dijkstra’s algorithm performance

We implemented a generic version of Dijkstra’s algorithm for the single-source shortest path problem corresponding to `Dijkstra4` in Fig. 1. This algorithm was selected since it allows to be executed both with a priority queue supporting decreasing keys (and no explicit keys in the items) and with a standard priority queue (with explicit keys in the items), and to study if allowing decreasing keys caused the number of comparisons performed to increase or decrease. We also tested what happens if we in addition to removing the explicit keys from the items also removed the visited array from the algorithm. This further reduces the space requirement for the algorithm, but breaks the $O(|E| \log |V|)$ running time guarantee.

Our function implementing `Dijkstra4` takes as arguments the graph, the priority queue to be used, and two Boolean flags `use_visited` and `use_dist`. If `use_visited` is false, the algorithm will not check if extracted nodes from the priority queue have been visited before, i.e., we could save the space for having a bit-vector for the visited nodes, the cost being that we might

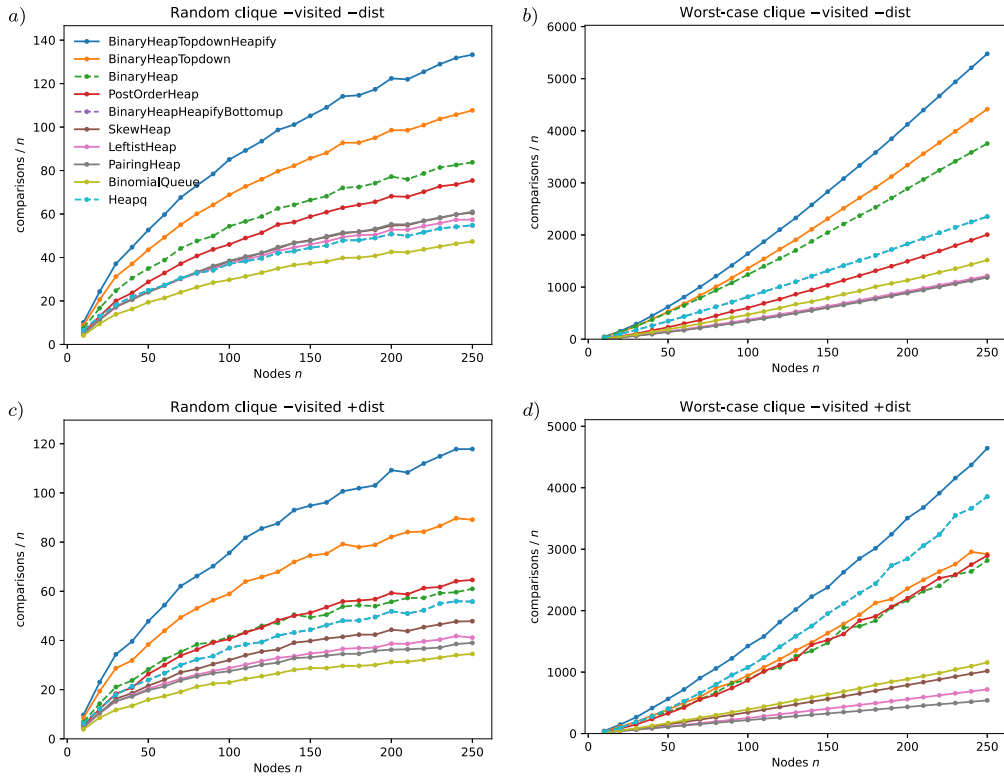
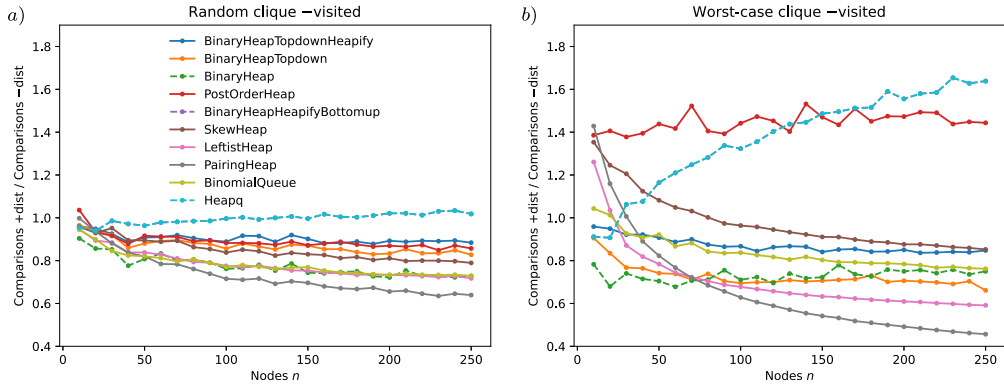


Fig. 10. Dijkstra, comparisons performed.

Fig. 11. Dijkstra, number of comparisons performed when using *dist* as key divided by the number of comparisons performed when storing explicit keys in the priority queues.

revisit nodes (and relax their outgoing edges) multiple times (once for each shorter distance discovered to the node). If `use_dist` is true, $dist[v]$ is used as the key of v when comparing v , otherwise an item in the priority queue consists of a pair $\langle \text{distance}, \text{node} \rangle$.

As input we tested directed cliques (including self loops) with n nodes and n^2 edges, where $10 \leq n \leq 250$. We did not consider other graph classes. We chose to consider cliques since they are the most dense graphs, potentially causing the largest number of decrease keys per node. We considered two types of edge weights: random integer weights from $1, \dots, n$, and weights forcing the worst-case number of key decreases. Node 0 is the source node. In the latter case, the edge from node u to v , for $0 \leq u, v \leq n-1$, has weight 1 if $v = u + 1$, weight 0 if $v \leq u$, and weight $n(n-u) + \sigma_u(v-u-2)$ otherwise, where σ_u is a random permutation of $\{0, 1, \dots, n-u-3\}$. The path $0 \xrightarrow{1} 1 \xrightarrow{1} \dots \xrightarrow{1} v-1 \xrightarrow{1} v$ is the shortest path to node v with distance v .

In the experiments we measured the number of comparisons performed by the priority queues, the number of nodes inserted into the priority queue, the number of nodes visited, and the number of edges relaxed. Each data point is the average over 10 random inputs. We do not report on the number of edges relaxed and nodes visited, since for cliques the number of edges relaxed is exactly n times the number of nodes visited, and each node is visited exactly once when using a visited bit-vector. Without a visited bit-vector each node insertion into the priority queue also causes the node to be visited once.

The experimental results are summarized in Figs. 9–11, where different combinations of `use_visited` and `use_dist` were tested. E.g., “+visited -dist” is when `use_visited` is true and `use_dist` is false, i.e., Dijkstra_4 without decreasing keys. The y -axis in Figs. 9 and 10 is the measured cost divided by n , i.e., the average cost per node.

Fig. 9 shows the number of insertions into the priority queue when not using a visited bit-vector. For random edge weights, Fig. 9(a,c), the probability is at most $1/k$ that the k 'th edge considered with node v as target decreases the distance to v (this follows by a simple backwards analysis argument), i.e., the expected number of times node v is inserted into the priority queue is at most $\sum_{k=1}^n \frac{1}{k} \approx \ln n$. This explains the nature of the curves in Fig. 9(a), where the priority queue stores $\langle \text{distance}, \text{node} \rangle$ pairs. When using `dist` as key, it follows from Section 2 that the same applies to the priority queues supporting decreasing keys, the non-dashed curves in Fig. 9(c). For cliques with worst-case edge weights, each edge (u, v) , where $u < v$, will cause a new shorter distance to v to be discovered, causing the number of insertions into the priority queue to be exactly $1 + \binom{n}{2}$. This explains the linear curves in Fig. 9(b,d).

Priority queues not supporting decreasing keys (Heapq, BinaryHeap and BinaryHeapHeapifyBottomup) might return the wrong items when using the current `dist` as keys. In combination with using a visited bit-vector, the algorithm might fail to compute the correct shortest distances. If the visited bit-vector is not used, then incorrect nodes returned by the priority queue will just be revisited later with a shorter distance, so the algorithm will still correctly find shortest paths—but there is no guarantee on the number of edge relaxes performed (except for an exponential upper bound that holds for any relax based approach). The increase in the number priority queue inserts appears neglectable for random weighted cliques (the dashed curves in Fig. 9(c) versus Fig. 9(a)), whereas for worst-case weights the number of insertions into the priority queue increases by 60–90 %, cf. Fig. 9(d) versus Fig. 9(b).

Finally, the number of comparisons performed in the priority queues when not using a visited bit-vector is depicted in Fig. 10 for the two types of edge weights and when using `dist` as key or not. In Fig. 10(a,b) the curve for SkewHeap is not very visible, since it essentially coincides with the PairingHeap curve. We have omitted all plots concerning experiments using a visited bit-vector. As mentioned above, the shortest distances might not be found for the combination using a visited bit-vector, `dist` as key, and a priority queue not supporting decreasing keys. For all priority queues supporting decreasing keys the sequence of priority queue operations is exactly the same when using a visited bit-vector and when not using it. The only difference is that using a visited bit-vector prevents the repeated relaxing of the same edge, where only the first relax can obtain a shorter distance.

The relative order on the number of comparisons for the difference priority queues on random weighted cliques in Fig. 10(a) is consistent with the ordering observed for sorting random integers in Fig. 8(a). The relative ordering for priority queues supporting decreasing keys is roughly unchanged when using `dist` as key or not (solid curves in Fig. 10(c) versus Fig. 10(a)). For worst-case weighted cliques the story is roughly the same, except that the increased number of priority queue insertions for Heapq when using `dist` as key (Fig. 9(d)) also increases the comparison count (Fig. 10(d) versus Fig. 10(b)). Interestingly, for BinaryHeap the significant increase in the number of priority queue insertions results in a smaller number of comparisons.

Interestingly, the number of comparisons performed appears in many cases to be a little bit lower if we use `dist` as key, cf. Fig. 10 (a) versus (c), and (b) versus (d). In Fig. 11 we have plotted the number of comparisons performed when using `dist` as the key relative to using $\langle \text{distance}, \text{node} \rangle$ pairs. The gain is largest for random cliques where the number of comparisons is reduced by typically at least 10 %. For worst-case weights the gain/loss varies heavily among the priority queues. It appears that *using a priority queue supporting decreasing keys one can often both save the space for storing explicit keys in the priority queue and reduce the number of comparisons performed*.

8. Conclusion

We have considered using priority queues supporting decreasing keys in Dijkstra's single source shortest path algorithm, motivated by the idea of saving space by omitting explicit keys for the items in the priority queue. Although standard binary heaps with bottom-up insertion fail to support decreasing keys, many other priority queues have been identified to do so, and in particular post-order heaps have been identified as a strong contender for a good alternative implicit priority queue in this context.

An open problem is to do a detailed experimental evaluation of the priority queues supporting decreasing keys in a low-level programming language like C. This is beyond the scope of this paper. Optimizing the running time of such implementations would require to carefully tune the code to take into account, e.g., caching, paging, branch mispredictions, and exploiting SIMD instructions [20]. Since the cache performance of binary heaps is known to be improvable by increasing the degree of the heap [16], one should also consider post-order heaps of higher degree.

CRedit authorship contribution statement

Gerth Stølting Brodal: Writing – original draft.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The author wants to thank Rolf Fagerberg for insightful discussions.

Funding

The work is supported by “Independent Research Fund Denmark”.

References

- [1] Georgy Adelson-Velsky, Evgenii Landis, An algorithm for the organization of information, *Proc. Acad. Sci. USSR* 146 (1962) 263–266 (in Russian).
- [2] Gerth Støtting Brodal, A survey on priority queues, in: Andrej Brodnik, Alejandro López-Ortiz, Venkatesh Raman, Alfredo Viola (Eds.), *Space-Efficient Data Structures, Streams, and Algorithms - Papers in Honor of J. Ian Munro on the Occasion of His 66th Birthday*, in: *Lecture Notes in Computer Science*, vol. 8066, Springer, 2013, pp. 150–163.
- [3] Gerth Støtting Brodal, Priority queues with decreasing keys, in: *11th International Conference on Fun with Algorithms, FUN 2022, May 30 to June 3, 2022, Island of Favignana, Sicily, Italy*, in: *LIPICs*, vol. 226, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022, pp. 8:1–8:19.
- [4] Svante Carlsson, J. Ian Munro, Patricio V. Poblete, An implicit binomial queue with constant insertion time, in: Rolf G. Karlsson, Andrzej Lingas (Eds.), *SWAT 88, 1st Scandinavian Workshop on Algorithm Theory, Halmstad, Sweden, July 5-8, 1988, Proceedings*, in: *Lecture Notes in Computer Science*, vol. 318, Springer, 1988, pp. 1–13.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022, <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>.
- [6] Clark A. Crane, *Linear Lists and Priority Queues as Balanced Binary Trees*, PhD thesis, Department of Computer Science, Stanford University, 1972.
- [7] Edsger W. Dijkstra, A note on two problems in connexion with graphs, *Numer. Math.* 1 (1959) 269–271, <https://doi.org/10.1007/BF01386390>.
- [8] James R. Driscoll, Harold N. Gabow, Ruth Shrairman, Robert Endre Tarjan, Relaxed heaps: an alternative to Fibonacci heaps with applications to parallel computation, *Commun. ACM* 31 (11) (1988) 1343–1354, <https://doi.org/10.1145/50087.50096>.
- [9] Amr Elmasry, Claus Jensen, Jyrki Katajainen, Two skew-binary numeral systems and one application, *Theory Comput. Syst.* 50 (1) (2012) 185–211, <https://doi.org/10.1007/s00224-011-9357-0>.
- [10] Michael L. Fredman, Robert Sedgwick, Daniel Dominic Sleator, Robert Endre Tarjan, The pairing heap: a new form of self-adjusting heap, *Algorithmica* 1 (1) (1986) 111–129, <https://doi.org/10.1007/BF01840439>.
- [11] Michael L. Fredman, Robert Endre Tarjan, Fibonacci heaps and their uses in improved network optimization algorithms, *J. ACM* 34 (3) (1987) 596–615, <https://doi.org/10.1145/28869.28874>.
- [12] Leonidas J. Guibas, Robert Sedgwick, A dichromatic framework for balanced trees, in: *19th Annual Symposium on Foundations of Computer Science, Ann Arbor, Michigan, USA, 16-18 October 1978, IEEE Computer Society, 1978*, pp. 8–21.
- [13] Nicholas J.A. Harvey, Kevin C. Zatloukal, The post-order heap, in: *Proceedings Third International Conference on Fun with Algorithms (FUN 2004), 2004*, <http://people.csail.mit.edu/nickh/Publications/PostOrderHeap/FUN04-PostOrderHeap.pdf>.
- [14] Vojtěch Jarník, O jistém problem minimálním, *Práce Moravské Pridovedecké Společnosti v Brně* 4 (1930) 57–63.
- [15] Irit Katriel, Ulrich Meyer, Elementary graph algorithms in external memory, in: Ulrich Meyer, Peter Sanders, Jop F. Sibeyn (Eds.), *Algorithms for Memory Hierarchies, Advanced Lectures*, in: *Lecture Notes in Computer Science*, vol. 2625, Springer, 2002, pp. 62–84.
- [16] Anthony LaMarca, Richard E. Ladner, The influence of caches on the performance of heaps, *ACM J. Exp. Algorithmics* 1 (4) (1996), <https://doi.org/10.1145/235141.235145>.
- [17] Eugene W. Myers, An applicative random-access stack, *Inf. Process. Lett.* 17 (5) (1983) 241–248, [https://doi.org/10.1016/0020-0190\(83\)90106-0](https://doi.org/10.1016/0020-0190(83)90106-0).
- [18] Thomas Porter, István Simon, Random insertion into a priority queue structure, *IEEE Trans. Softw. Eng.* 1 (3) (1975) 292–298, <https://doi.org/10.1109/TSE.1975.6312854>.
- [19] R.C. Prim, Shortest connection networks and some generalizations, *Bell Syst. Tech. J.* 36 (6) (1957) 1389–1401, <https://doi.org/10.1002/j.1538-7305.1957.tb01515.x>.
- [20] Peter Sanders, Fast priority queues for cached memory, *ACM J. Exp. Algorithmics* 5 (7) (2000), <https://doi.org/10.1145/351827.384249>.
- [21] Daniel Dominic Sleator, Robert Endre Tarjan, Self-adjusting heaps, *SIAM J. Comput.* 15 (1) (1986) 52–69, <https://doi.org/10.1137/0215004>.
- [22] Jean Vuillemin, A data structure for manipulating priority queues, *Commun. ACM* 21 (4) (1978) 309–315, <https://doi.org/10.1145/359460.359478>.
- [23] J.W.J. Williams, Algorithm 232 heapsort, *Commun. ACM* 7 (6) (1964) 347–348, <https://doi.org/10.1145/512274.512284>.